

1. Innovation and Novelty

- **Action Invertibility:** We developed an automated pipeline to inject 6D inverse-kinematics physical noise into expert demonstrations and mathematically invert the actions to gracefully recover. This introduces critical out-of-distribution (OOD) states, forcing downstream policies to learn robust recovery behaviors rather than simply cloning perfect, deterministic trajectories.
- **Experimentation:** Our innovation is backed by rigorous experimentation. We successfully validated the invertibility hypothesis in the MuJoCo simulator, handled complex state-dimensionality shifts across suites (e.g., 110-dim to 79-dim), and resolved low-level OpenGL/EGL rendering artifacts (the 180-degree camera flip issue) to ensure perfect visual fidelity.

1. Innovation and Novelty

- **Trajectory-Level Perturbation:** State-of-the-art benchmarks like LIBERO-Plus reveal that current VLAs fail drastically on initial state distributions (performance collapsing from >95% to <30%). While existing methodologies focus purely on scene-level augmentations (lighting, textures), we uniquely address this by perturbing the robot's actual state trajectory. This algorithmically expands the dataset to previously unseen, physically valid configurations. Furthermore, this structured trajectory expansion unlocks immense potential for future applications in curriculum learning and RL-based few-shot fine-tuning.
- **Standardization & LeRobot Compliance:** Most manipulation datasets are still isolated in native, complex formats like HDF5. We architected a highly parallelized conversion pipeline to map the LIBERO state vectors directly into the Hugging Face LeRobot v3.0 format (Parquet + SVT-AV1 compressed video). This makes the dataset universally accessible and "plug-and-play" ready, making it extremely easy to use in any modern VLA development pipeline.

Final Presentation

1. Innovation and Novelty – Action Invertibility

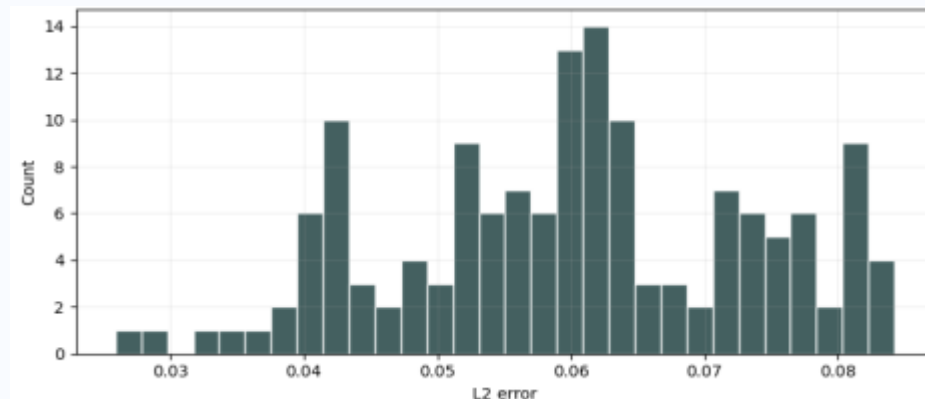
Action Reversibility Validation

Can we recover state s_1 by applying the inverse action $-a_1$ from s_2 ?

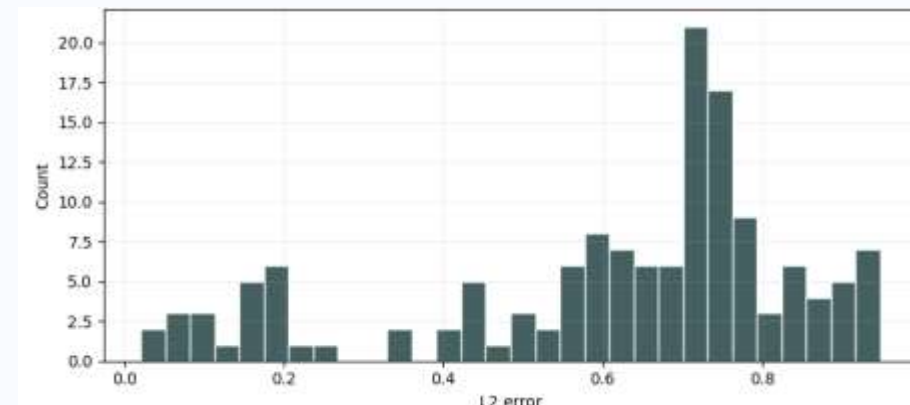
- **Simulator replay:** mean error 0.060 (std 0.013)
- **Naive vector addition:** mean error 0.613 (std 0.239)

Simulator replay yields nearly 10× smaller error, keeping the robot within millimeters of the original path. One reverse step safely rejoins the demonstrated trajectory.

Histograms (below): Full error distributions for both methods.



Simulator Replay



Naive vector addition

1. Innovation and Novelty – Impact

Impact

- While our published dataset provides **13,000 newly augmented trajectories** (2 augmentations per expert demonstration), our open-source codebase is entirely parameterized. Researchers can seamlessly scale this to generate an unlimited number of unique OOD recovery trajectories per expert demo to fit their specific training scale and compute budget.
- Our augmentation is **model-agnostic**. It can be applied on top of any existing VLA training pipeline and combined with scene-level augmentation. This lowers the barrier for the community to build more robust policies.
- By demonstrating that trajectory diversity is a missing ingredient in current augmentation, we hope to shift how training data is prepared for manipulation tasks.

2. Architecture & Open Datasets

Source Datasets

We built upon the standard open-source LIBERO HDF5 datasets, leveraging the native expert demonstrations for the `libero_goal`, `libero_spatial`, and `libero_object` benchmark suites.

Dataset Link: <https://huggingface.co/datasets/yifengzhu-hf/LIBERO-datasets>

2. Architecture & Open Datasets

Method details

- **Timestep selection:** Find the last index before gripper closure across all trajectories, and sample earlier indices with higher probability to maximize variation while preserving task semantics.
- **Noise injection:** Sample a small Gaussian noise action ϵ , scaled to the average action magnitude in the dataset. Apply ϵ in the simulator from state s_i to reach a new, physically valid state s_{new} .
- **Trajectory construction:** From s_{new} , apply the inverse action $-\epsilon$ in the simulator to step back near s_i (0.06 error). Then continue with the original actions $a_i, a_{i+1}, \dots$ to complete the task.
- **Validation:** Each generated trajectory is executed in the LIBERO simulator. Only episodes that successfully complete the task are kept. This guarantees collision-free, physically valid data.

2. Architecture & Open Datasets – How do we achieve the methodology?

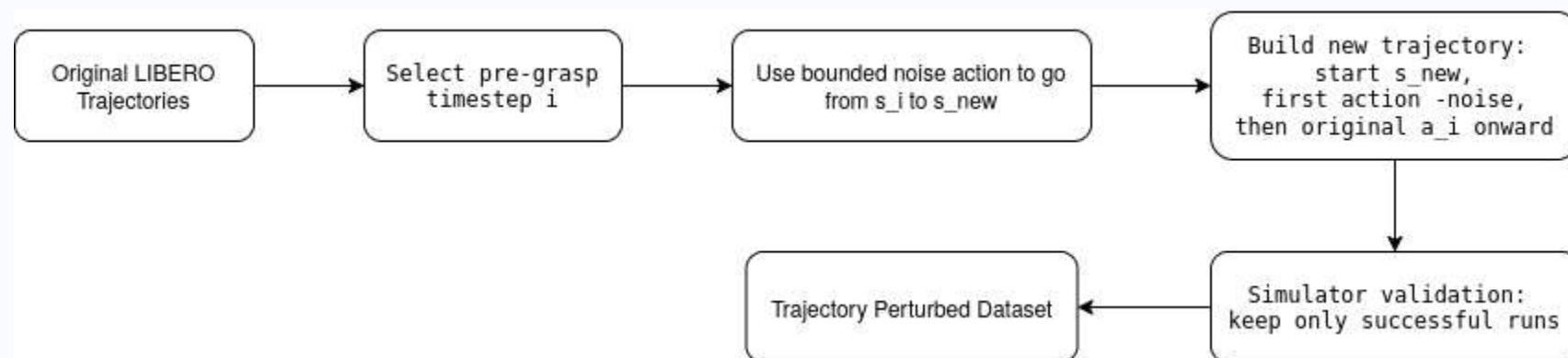
Architectural Workflow:

1. Parse original HDF5 demonstrations to locate critical pre-grasp timesteps.
2. Inject bounded, physically valid Gaussian noise using a headless MuJoCo simulator.
3. Step back toward the original path using the inverse action (reducing reconstruction error drastically from 0.613 with naive math to 0.060 with simulator replay).
4. Convert the robust, augmented trajectories into the LeRobot format for high-throughput VLA training.

Final Presentation

2. Architecture & Open Datasets

Augmentation Pipeline:



2. Architecture & Open Datasets

Why LeRobot?

Building Infrastructure that is easy to use!

LeRobot is the rapidly emerging gold standard in the Hugging Face robotics ecosystem. By standardizing our output to LeRobot, our dataset integrates natively with Hugging Face's LeRobotDataset API, enabling instant data loading for architectures like SmolVLA, ALOHA, and ACT.

3. Final Deliverables

- **The Dataset:** A fully augmented, LeRobot v3-compatible dataset encompassing multiple LIBERO task suites, designed specifically to train robust recovery behaviors, containing 13,000 newly augmented trajectories. We also provide parametrized code allowing researchers to **seamlessly scale this to generate an unlimited number of unique OOD recovery trajectories per expert demo** to fit their specific training scale and compute budget.
- **Dataset Link:** https://huggingface.co/datasets/jdhr/libero_trajectory_augmented
- **Repository Link:** https://github.com/JDhruvR/Trajectory_Augmentation

4. Future Work

- **Training & Evaluation Scope:** The natural next step is to fine-tune compact VLAs (such as SmolVLA, OpenVLA, etc) exclusively on this augmented dataset to quantify trajectory-diversity gains without requiring new human demonstrations.
- **Targeting Benchmarks:** By evaluating on robust splits like LIBERO-Plus and LIBERO-PRO, future studies can concretely measure the reduction in brittleness when models face mid-trajectory disturbances and unseen robotic start configurations.

4. Future Work

- **Curriculum Learning Framework:** We can develop a curriculum-based training pipeline where VLAs are initially taught using easier, shorter trajectories originating near the target grasp point. As the agent's success rate improves, we can incrementally introduce trajectories starting further away, eventually chaining them with perturbations of increasing magnitude (e.g., perturbing from an already perturbed state).
- **Sim-to-Real (Sim2Real) Transfer Validation:** Real-world robotic deployments suffer heavily from motor jitter, sensor noise, and slight spatial discrepancies. Because our trajectory augmentation physically simulates these exact types of noise, a critical future step is deploying VLAs trained on our dataset onto physical robotic arms (e.g., a physical Franka Panda) to see if this method effectively closes the sim-to-real gap.

Final Presentation

5. AI & Agentic Tool Usage

We relied heavily on AI coding assistants through agentic interfaces like **Antigravity** and **AlphaXiv** to accelerate development. The LLMs we chose for our agents were primarily - Gemini 3.1 Pro (High) and Claude Opus 4.6 (Thinking).

Agentic Coding:

- **Antigravity/Opencode:** Used for general coding and rapid iteration.

Research:

- **AlphaXiv:** Used for AI-assisted paper review and evidence extraction.

Agent Skills (from github.com/mattpocock/skills):

- Some of the skills we use most are `grill-with-docs`, `tdd`, `improve-codebase-architecture`, `zoom-out`, `caveman`, `grill-me`, `git-guardrails`

5. AI & Agentic Tool Usage

Reasoning & Planning:

The most successful workflow we discovered was the **Plan-Then-Execute** pattern. By forcing the agent to generate a structured implementation plan and wait for human approval before writing code, we prevented it from going down counterproductive rabbit holes.

5. AI & Agentic Tool Usage

What Worked?

- **Asynchronous Tool Chaining:** The agent flawlessly orchestrated terminal commands, spun up tmux sessions, and handled long-running batch conversions (processing ~4,500 trajectories) autonomously in the background.
- **Stack Trace Parsing:** The agent excelled at deciphering cryptic MuJoCo and Robosuite stack traces, saving hours of manual debugging.
- **Dynamic Code Exploration:** The agent autonomously navigated the dense LIBERO source code to discover undocumented state dimensionality changes across suites without explicit human guidance.
- **Agent Skills Integration:** We utilized strict behavioral prompts (adapted from mattpocock/skills) like `tdd` for test-driven development, `improve-codebase-architecture` for modularity, and `caveman` to dramatically compress token overhead. We intentionally kept our `AGENT.md` guidelines under 150 lines to prevent context degradation.

5. AI & Agentic Tool Usage

What Didn't Work?

- **Math Across Files:** The agent struggled with tracking mathematical operations (like our 180-degree image flip bug) across multiple scripts, initially attempting to apply FFmpeg band-aids rather than fixing the root matrix math in the rendering pipeline.
- **Sycophancy & Rabbit Holes:** The agent occasionally exhibited "sycophantic" loops. Blindly repeating failed bash commands to please the user instead of stepping back to diagnose the failure. It also struggled to "zoom out" of failing approaches without explicit human overrides.
- **Context Loss:** Over long sessions, the agent would occasionally forget global rules (like always using headless eg1 rendering), which required us to track engineering decisions in dedicated markdown files as a traceable record.
- **Spontaneous Commits:** The agent would occasionally commit changes to git without asking, requiring strict behavioral controls via `git-guardrails` to block dangerous operations before execution.

5. AI & Agentic Tool Usage

Takeaway:

Agentic coding tools dramatically sped up our development. However, the best results came from treating the agent as a highly capable engineer but extremely convoluted in its vision: excellent at execution, but requiring strict guardrails, architectural direction, and human check-ins to stay on track.

5. AI & Agentic Tool Usage – Additional Info

Agent Skills Overview:

- **grill-with-docs:** Aligns the agent and developer by building a shared vocabulary and recording architecture decisions in documents.
- **tdd:** Writes a failing test first, then the code to pass it, so every change is verified immediately.
- **improve-codebase-architecture:** Regularly restructures code to keep it clean, modular, and easy to change.
- **zoom-out:** Explains how a piece of code fits into the whole system before making edits, avoiding narrow fixes that break other parts.
- **caveman:** Compresses agent messages to remove filler, cutting token usage by about 75% without losing technical accuracy.
- **grill-me:** Relentlessly questions a plan until every detail and edge case is resolved.
- **git-guardrails:** Blocks dangerous git operations like `git push --force` and `git reset --hard` before they execute.

6. References

- LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning
arXiv:2306.03310v2
- LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models
arXiv:2510.13626v3
- LIBERO-PRO: Towards Robust and Fair Evaluation of Vision-Language-Action Models Beyond Memorization
arXiv:2510.03827v1
- LeRobotDataset v3.0 Documentation
<https://huggingface.co/docs/lerobot/lerobot-dataset-v3>
- robosuite: A Modular Simulation Framework and Benchmark for Robot Learning
arXiv:2009.12293